KATEDRA
INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI

Paradigmata programování 1

9 Lexikální uzávěry

Michal Krupka

Obsah

1 Co víme o uživatelsky definovaných funkcích

2 Lexikální uzávěr

3 Anonymní funkce

4 Posloupnosti

5 Dynamické vytváření nových funkcí

6 Příklad: bankovní účet

Otázka z minula

```
(defun scale-list (list factor)
  (labels ((prod (x) (* x factor)))
    (mapcar #'prod list)))
```

Otázka z minula

```
(defun scale-list (list factor)
  (labels ((prod (x) (* x factor)))
    (mapcar #'prod list)))
```

Jak to, že to fuguje?

Definice funkce

Definice funkce

- spec. operátorem `defun` nebo `labels`.

Definice funkce

- spec. operátorem `defun` nebo `labels`.

Zadává se:

- jméno
- seznam parametrů
- tělo

Definice funkce

- spec. operátorem `defun` nebo `labels`.

Zadává se:

- jméno
- seznam parametrů
- tělo

Funkce si pamatuje svůj seznam parametrů a tělo.

Aplikace uživatelské funkce

Aplikace uživatelské funkce

Aplikace uživatelské funkce

Při aplikaci uživatelské funkce se vytvoří nové prostředí s vazbami parametrů na argumenty. Předkem tohoto prostředí se stane globální prostředí. V tomto prostředí se pak vyhodnotí tělo funkce. Nové prostředí se vytváří při každé aplikaci znovu.

Za chvíli tento popis upravíme.

Obsah

1 Co víme o uživatelsky definovaných funkcích

2 Lexikální uzávěr

3 Anonymní funkce

4 Posloupnosti

5 Dynamické vytváření nových funkcí

6 Příklad: bankovní účet

Informace ukrytá ve funkci

Jak to, že příklady z minulé přednášky fungovaly?

Informace ukrytá ve funkci

Jak to, že příklady z minulé přednášky fungovaly?

Např. funkce `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Informace ukrytá ve funkci

Jak to, že příklady z minulé přednášky fungovaly?

Např. funkce `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Výrazy `(multiplier 5)` a `(multiplier 10)` se oba vyhodnotí na funkci. Přitom

Informace ukrytá ve funkci

Jak to, že příklady z minulé přednášky fungovaly?

Např. funkce `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Výrazy `(multiplier 5)` a `(multiplier 10)` se oba vyhodnotí na funkci. Přitom

- Seznam parametrů obou funkcí je `(x)`,
- tělo obou funkcí je `(* x factor)`.

Informace ukrytá ve funkci

Jak to, že příklady z minulé přednášky fungovaly?

Např. funkce `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Výrazy `(multiplier 5)` a `(multiplier 10)` se oba vyhodnotí na funkci. Přitom

- Seznam parametrů obou funkcí je `(x)`,
- tělo obou funkcí je `(* x factor)`.

Funkce tedy mají stejný seznam parametrů a stejné tělo. Přesto vracejí různé výsledky.

Informace ukrytá ve funkci

Jak to, že příklady z minulé přednášky fungovaly?

Např. funkce `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Výrazy `(multiplier 5)` a `(multiplier 10)` se oba vyhodnotí na funkci. Přitom

- Seznam parametrů obou funkcí je `(x)`,
- tělo obou funkcí je `(* x factor)`.

Funkce tedy mají stejný seznam parametrů a stejné tělo. Přesto vracejí různé výsledky.

Obsahují tedy nějakou informaci navíc.

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

- Vyhodnocení výrazu `(multiplier 5)`

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

- Vyhodnocení výrazu `(multiplier 5)`

  Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

- Vyhodnocení výrazu `(multiplier 5)`
  Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

- Vyhodnocení výrazu `(multiplier 5)`
  Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

funkce `funcall` aplikuje funkci `prod` na 10:

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

Vyhodnocení výrazu `(multiplier 5)`

Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

funkce `funcall` aplikuje funkci `prod` na 10:

Aplikace `prod` na 10

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

Vyhodnocení výrazu `(multiplier 5)`

Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

funkce `funcall` aplikuje funkci `prod` na 10:

Aplikace `prod` na 10

Vytvoří se prostředí *env* s vazbou symbolu `x` na 10

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

Vyhodnocení výrazu `(multiplier 5)`

Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

funkce `funcall` aplikuje funkci `prod` na 10:

Aplikace `prod` na 10

Vytvoří se prostředí *env* s vazbou symbolu `x` na 10

v něm se vyhodnotí tělo:

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

Vyhodnocení výrazu `(multiplier 5)`

Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

funkce `funcall` aplikuje funkci `prod` na 10:

Aplikace `prod` na 10

Vytvoří se prostředí *env* s vazbou symbolu `x` na 10

v něm se vyhodnotí tělo:

Vyhodnocení výrazu `(* x factor)` v prostředí *env*

Informace ukrytá ve funkci

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vyhodnocení výrazu `(funcall (multiplier 5) 10)`

`funcall` je funkce, vyhodnotí se tedy argumenty:

Vyhodnocení výrazu `(multiplier 5)`

Výsledkem je funkce `prod` se seznamem parametrů `(x)` a tělem `(* x factor)`

druhý argument má hodnotu 10

funkce `funcall` aplikuje funkci `prod` na 10:

Aplikace `prod` na 10

Vytvoří se prostředí *env* s vazbou symbolu `x` na 10

v něm se vyhodnotí tělo:

Vyhodnocení výrazu `(* x factor)` v prostředí *env*

Kde se hledá hodnota symbolu `factor`?

Prostředí vzniku funkce

Prostředí vzniku funkce

- U operátoru `defun` je to prostředí, ve kterém byl výraz s operátorem `defun` vyhodnocen (obvykle je to tedy globální prostředí), tj. aktuální prostředí.
- U operátoru `labels` je to prostředí tímto operátorem vytvořené (jehož je, jak víme, aktuální prostředí předkem).

Prostředí vzniku hodnoty `(multiplier 5)`

Globální prostředí

symbol	hodnota	typ vazby
`multiplier`	`#<FUNCTION ...>`	*funkční*
`pi`	`3.141...`	*hodnotová*
⋮	⋮	⋮

Prostředí funkce `multiplier`

symbol	hodnota	typ vazby
`factor`	5	*hodnotová*

Prostředí operátoru `labels`

symbol	hodnota	typ vazby
`prod`	`#<LOCAL-FUNCTION PROD ... >`	*funkční*

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Lexikální uzávěr

každá funkce obsahuje následující informace

- seznam parametrů,
- tělo,
- prostředí vzniku.

Lexikální uzávěr

každá funkce obsahuje následující informace

- seznam parametrů,
- tělo,
- prostředí vzniku.

Aplikace uživatelské funkce

Při aplikaci uživatelské funkce se vytvoří nové prostředí s vazbami parametrů na argumenty. Předkem tohoto prostředí se stane prostředí vzniku funkce. V tomto prostředí se pak vyhodnotí tělo. Nové prostředí se vytváří při každé aplikaci znovu.

Lexikální uzávěr

každá funkce obsahuje následující informace

- seznam parametrů,
- tělo,
- prostředí vzniku.

Aplikace uživatelské funkce

Při aplikaci uživatelské funkce se vytvoří nové prostředí s vazbami parametrů na argumenty. Předkem tohoto prostředí se stane prostředí vzniku funkce. V tomto prostředí se pak vyhodnotí tělo. Nové prostředí se vytváří při každé aplikaci znovu.

Funkce, která nese informaci o prostředí svého vzniku (v Lispu je to každá), se nazývá *lexikální uzávěr* (stručně *uzávěr*).

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.

Globální prostředí

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

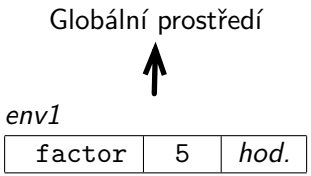
Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.

Globální prostředí

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

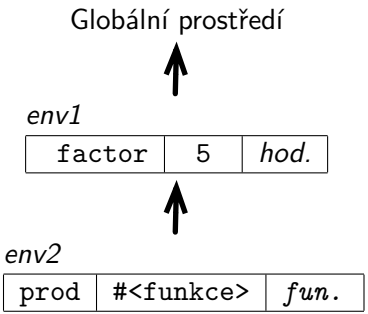
Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.




```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

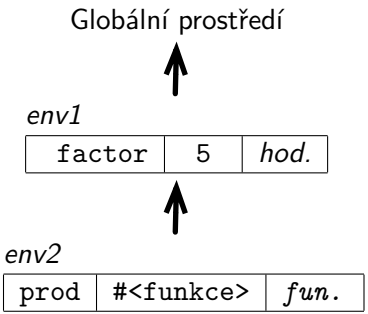
Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.




```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

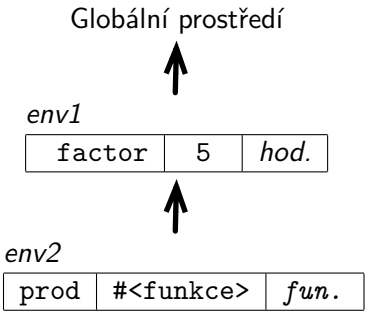



```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

Seznam parametrů: `(x)`
Tělo: `(* x factor)`
Prostředí vzniku: *env2*



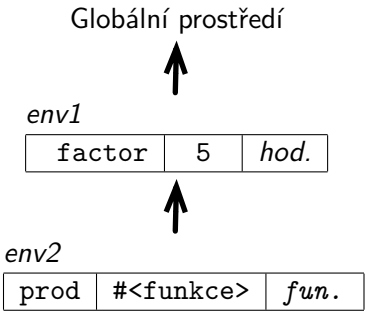

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

Seznam parametrů: `(x)`
Tělo: `(* x factor)`
Prostředí vzniku: *env2*

Funkce `prod` se pak aplikuje na 10.



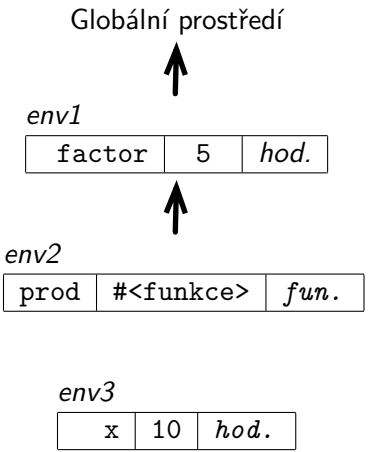

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

Seznam parametrů: `(x)`
Tělo: `(* x factor)`
Prostředí vzniku: *env2*

Funkce `prod` se pak aplikuje na `10`.
Při aplikaci se vytvoří prostředí *env3*.



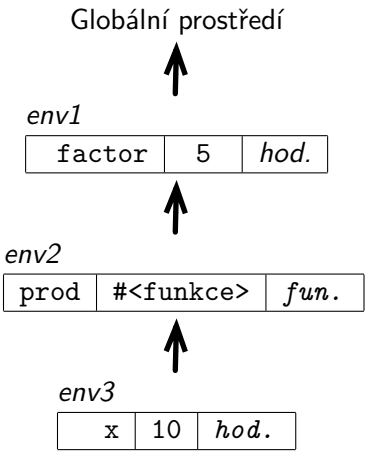

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

Seznam parametrů: `(x)`
Tělo: `(* x factor)`
Prostředí vzniku: *env2*

Funkce `prod` se pak aplikuje na `10`.
Při aplikaci se vytvoří prostředí *env3*.
Jeho předkem se stane prostředí vzniku, tedy *env2*.



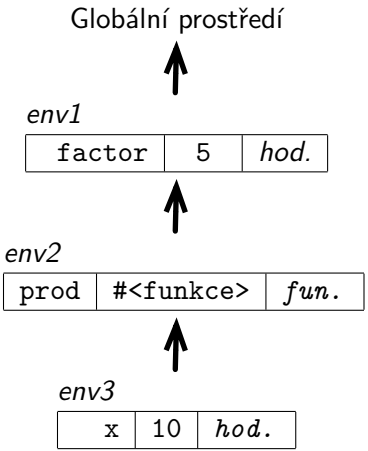

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

Seznam parametrů: `(x)`
Tělo: `(* x factor)`
Prostředí vzniku: *env2*

Funkce `prod` se pak aplikuje na `10`.
Při aplikaci se vytvoří prostředí *env3*.
Jeho předkem se stane prostředí vzniku, tedy *env2*.
V *env3* se vyhodnotí výraz `(* x factor)`.



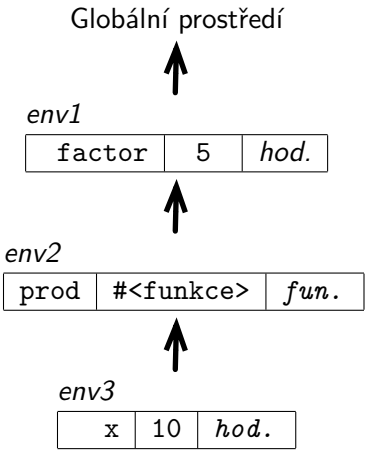

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Prostředí při vyhodnocování `(funcall (multiplier 5) 10)`

Výraz se vyhodnocuje v globálním prostředí.
Jako první se vyhodnocuje `(multiplier 5)`.
Při aplikaci se vytvoří prostředí *env1*.
Operátor `labels` vytvoří prostředí *env2*.
Vznikne lokální funkce `prod`, která si pamatuje:

Seznam parametrů: `(x)`
Tělo: `(* x factor)`
Prostředí vzniku: *env2*

Funkce `prod` se pak aplikuje na `10`.
Při aplikaci se vytvoří prostředí *env3*.
Jeho předkem se stane prostředí vzniku, tedy *env2*.
V *env3* se vyhodnotí výraz `(* x factor)`.
Výsledek: 50.




```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Obsah

1 Co víme o uživatelsky definovaných funkcích

2 Lexikální uzávěr

3 Anonymní funkce

4 Posloupnosti

5 Dynamické vytváření nových funkcí

6 Příklad: bankovní účet

Speciální operátor `lambda`

Speciální operátor `lambda`

Někdy nepotřebujeme zadávat jméno funkce.

Speciální operátor `lambda`

Někdy nepotřebujeme zadávat jméno funkce.

$$\texttt{(lambda } \overbrace{\texttt{(b h)}}^{\textit{seznam parametrů}} \ \underbrace{\texttt{(* 1/2 b h)}}_{\textit{tělo}\text{ funkce}}\texttt{)}$$

Použití

Použití

```
(defun multiplier (factor)
  (lambda (x) (* x factor)))
```

Použití

```
(defun multiplier (factor)
  (lambda (x) (* x factor)))
```

```
(defun scale-list (list factor)
  (mapcar (lambda (x) (* x factor))
          list))
```

```
(defun scale-list (list factor)
  (mapcar (multiplier factor)
          list))
```

Obsah

1 Co víme o uživatelsky definovaných funkcích

2 Lexikální uzávěr

3 Anonymní funkce

4 Posloupnosti

5 Dynamické vytváření nových funkcí

6 Příklad: bankovní účet

Sčítání prvků posloupnosti

Sčítání prvků posloupnosti

Součet celých čísel v intervalu:

Sčítání prvků posloupnosti

Součet celých čísel v intervalu:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ from (sum-numbers (+ from 1) to))))
```

Sčítání prvků posloupnosti

Součet celých čísel v intervalu:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ from (sum-numbers (+ from 1) to))))
```

Součet druhých mocnin:

Sčítání prvků posloupnosti

Součet celých čísel v intervalu:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ from (sum-numbers (+ from 1) to))))
```

Součet druhých mocnin:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (expt from 2) (sum-squares (1+ from) to))))
```

Sčítání prvků posloupnosti

Součet celých čísel v intervalu:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ from (sum-numbers (+ from 1) to))))
```

Součet druhých mocnin:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (expt from 2) (sum-squares (1+ from) to))))
```

Funkce sčítá členy *posloupnosti druhých mocnin přirozených čísel* (včetně nuly):

$$0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121 \ldots$$

Sčítání prvků posloupnosti

Součet celých čísel v intervalu:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ from (sum-numbers (+ from 1) to))))
```

Součet druhých mocnin:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (expt from 2) (sum-squares (1+ from) to))))
```

Funkce sčítá členy *posloupnosti druhých mocnin přirozených čísel* (včetně nuly):

$$0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121 \ldots$$

Čím se tyto funkce liší?

Reprezentace posloupností funkcemi (druhé mocniny)

Reprezentace posloupností funkcemi (druhé mocniny)

Pomocí funkce

```
(defun power2 (x)
  (* x x))
```

přepíšeme `sum-squares` takto:

Reprezentace posloupností funkcemi (druhé mocniny)

Pomocí funkce

```
(defun power2 (x)
  (* x x))
```

přepíšeme `sum-squares` takto:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (power2 from) (sum-squares (1+ from) to))))
```

Reprezentace posloupností funkcemi (druhé mocniny)

Pomocí funkce

```
(defun power2 (x)
  (* x x))
```

přepíšeme `sum-squares` takto:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (power2 from) (sum-squares (1+ from) to))))
```

Posloupnost druhých mocnin přirozených čísel je dána (*reprezentována*) funkcí `power2`.

Reprezentace posloupností funkcemi (přirozená čísla)

Reprezentace posloupností funkcemi (přirozená čísla)

Funkce `identity`:

```
(defun identity (x)
  x)
```

Reprezentace posloupností funkcemi (přirozená čísla)

Funkce `identity`:

```
(defun identity (x)
  x)
```

Přepsaná funkce `sum-numbers`:

Reprezentace posloupností funkcemi (přirozená čísla)

Funkce `identity`:

```
(defun identity (x)
  x)
```

Přepsaná funkce `sum-numbers`:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ (identity from) (sum-numbers (+ from 1) to))))
```

Reprezentace posloupností funkcemi (přirozená čísla)

Funkce `identity`:

```
(defun identity (x)
  x)
```

Přepsaná funkce `sum-numbers`:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ (identity from) (sum-numbers (+ from 1) to))))
```

Posloupnost přirozených čísel včetně nuly:

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, \ldots$$

lze zadat funkcí `identity`.

Reprezentace posloupností funkcemi (Fibonacciho posloupnost)

Reprezentace Fibonacciho posloupnosti funkcí `fib`:

```
(defun fib (n)
  (cond ((= n 0) 0)
        ((= n 1) 1)
        (t (+ (fib (- n 2)) (fib (- n 1))))))
```

(umíme zrychlit)

Součet členů obecné posloupnosti

Součet členů obecné posloupnosti

Přístup k členům posloupnosti:

Součet členů obecné posloupnosti

Přístup k členům posloupnosti:

```
(defun mem (seq index)
  (funcall seq index))
```

Součet členů obecné posloupnosti

Přístup k členům posloupnosti:

```
(defun mem (seq index)
  (funcall seq index))
```

Součet členů posloupnosti:

```
(defun sum-sequence (seq from to)
  (if (< to from)
      0
    (+ (mem seq from)
       (sum-sequence seq (+ from 1) to))))
```

Obsah

1 Co víme o uživatelsky definovaných funkcích

2 Lexikální uzávěr

3 Anonymní funkce

4 Posloupnosti

5 Dynamické vytváření nových funkcí

6 Příklad: bankovní účet

Převod posloupnosti na seznam

Převod posloupnosti na seznam

```
(defun seq-to-list (seq len)
  (labels ((stl (index)
             (if (>= index len)
                 '()
               (cons (mem seq index)
                     (stl (+ index 1))))))
    (stl 0)))
```

Konstantní posloupnosti

- *Konstantní posloupnost:* posloupnost, jejíž všechny členy jsou stejné

Konstantní posloupnosti

- *Konstantní posloupnost:* posloupnost, jejíž všechny členy jsou stejné

Například:

```
(defun constantly-10 (n)
  10)
```

Konstantní posloupnosti

- *Konstantní posloupnost:* posloupnost, jejíž všechny členy jsou stejné

Například:

```
(defun constantly-10 (n)
  10)
```

Obecně:

```
(defun constantly (c)
  (lambda (n) c))
```

Součet posloupností

Součet posloupností

Součtem dvou posloupností rozumíme posloupnost, jejíž členy jsou součty příslušných členů původních posloupností:

Součet posloupností

Součtem dvou posloupností rozumíme posloupnost, jejíž členy jsou součty příslušných členů původních posloupností:

```
(defun seq-+ (seq1 seq2)
  (lambda (n)
    (+ (mem seq1 n) (mem seq2 n))))
```

Součet posloupností

Součtem dvou posloupností rozumíme posloupnost, jejíž členy jsou součty příslušných členů původních posloupností:

```
(defun seq-+ (seq1 seq2)
  (lambda (n)
    (+ (mem seq1 n) (mem seq2 n))))
```

Posloupnost, jejíž člen o indexu n je dán předpisem

$$a_n = n^2 + 1,$$

je tedy reprezentována funkcí vzniklou vyhodnocením výrazu

```
(seq-+ #'power2 (constantly 1))
```

Operace s posloupnostmi

Operace s posloupnostmi

Obecná operace se dvěma posloupnostmi:

Operace s posloupnostmi

Obecná operace se dvěma posloupnostmi:

```
(defun seq-op (op seq1 seq2)
  (lambda (n)
    (funcall op (mem seq1 n) (mem seq2 n))))
```

Operace s posloupnostmi

Obecná operace se dvěma posloupnostmi:

```
(defun seq-op (op seq1 seq2)
  (lambda (n)
    (funcall op (mem seq1 n) (mem seq2 n))))
```

Aritmetický průměr dvou posloupností:

Operace s posloupnostmi

Obecná operace se dvěma posloupnostmi:

```
(defun seq-op (op seq1 seq2)
  (lambda (n)
    (funcall op (mem seq1 n) (mem seq2 n))))
```

Aritmetický průměr dvou posloupností:

```
(seq-op (lambda (x y) (/ (+ x y) 2)) seq1 seq2)
```

Hledání v posloupnosti

Hledání v posloupnosti

```
(defun first-index (seq condition)
  (labels ((iter (index)
             (if (funcall condition (mem seq index))
                 index
               (iter (1+ index)))))
    (iter 0)))
```

(Funkce se zacyklí, pokud žádný člen posloupnosti podmínku nesplňuje.)

Hledání v posloupnosti

```
(defun first-index (seq condition)
  (labels ((iter (index)
             (if (funcall condition (mem seq index))
                 index
               (iter (1+ index)))))
    (iter 0)))
```

(Funkce se zacyklí, pokud žádný člen posloupnosti podmínku nesplňuje.)

Kolikáté Fibonacciho číslo je větší než 100?

Hledání v posloupnosti

```
(defun first-index (seq condition)
  (labels ((iter (index)
             (if (funcall condition (mem seq index))
                 index
               (iter (1+ index)))))
    (iter 0)))
```

(Funkce se zacyklí, pokud žádný člen posloupnosti podmínku nesplňuje.)

Kolikáté Fibonacciho číslo je větší než 100?

```
(first-index #'fib (lambda (x) (> x 100)))
```

Hledání v posloupnosti

```
(defun first-index (seq condition)
  (labels ((iter (index)
             (if (funcall condition (mem seq index))
                 index
               (iter (1+ index)))))
    (iter 0)))
```

(Funkce se zacyklí, pokud žádný člen posloupnosti podmínku nesplňuje.)

Kolikáté Fibonacciho číslo je větší než 100?

```
(first-index #'fib (lambda (x) (> x 100)))
```

Kolikáté Fibonacciho číslo je dělitelné 14 (kromě nuly)?

Hledání v posloupnosti

```
(defun first-index (seq condition)
  (labels ((iter (index)
             (if (funcall condition (mem seq index))
                 index
               (iter (1+ index)))))
    (iter 0)))
```

(Funkce se zacyklí, pokud žádný člen posloupnosti podmínku nesplňuje.)

Kolikáté Fibonacciho číslo je větší než 100?

```
(first-index #'fib (lambda (x) (> x 100)))
```

Kolikáté Fibonacciho číslo je dělitelné 14 (kromě nuly)?

```
(first-index #'fib (lambda (x)
                     (and (> x 0)
                          (= (rem x 14) 0))))
```

Obsah

1 Co víme o uživatelsky definovaných funkcích

2 Lexikální uzávěr

3 Anonymní funkce

4 Posloupnosti

5 Dynamické vytváření nových funkcí

6 Příklad: bankovní účet

Vklady na účet

Vkládáme každý měsíc 10000 Kč na bankovní účet. Měsíce číslujeme od nuly, první vklad uděláme hned v měsíci č. 0 (při založení účtu).

Vklady na účet

Vkládáme každý měsíc 10000 Kč na bankovní účet. Měsíce číslujeme od nuly, první vklad uděláme hned v měsíci č. 0 (při založení účtu).

Posloupnost vkladů:

```
(constantly 10000.0)
```

Vklady na účet

Vkládáme každý měsíc 10000 Kč na bankovní účet. Měsíce číslujeme od nuly, první vklad uděláme hned v měsíci č. 0 (při založení účtu).

Posloupnost vkladů:

```
(constantly 10000.0)
```

Uložíme do proměnné:

Vklady na účet

Vkládáme každý měsíc 10000 Kč na bankovní účet. Měsíce číslujeme od nuly, první vklad uděláme hned v měsíci č. 0 (při založení účtu).

Posloupnost vkladů:

```
(constantly 10000.0)
```

Uložíme do proměnné:

```
> (bind deposits (constantly 10000.0))
#<ANONYMOUS-FUNCTION subfunction of CONSTANTLY>
```

Úročení

Úročení

Zůstatek:

$$B_0 = d_0$$
$$B_n = B_{n-1} \cdot (1 + i) + d_n \text{ pro } n > 0$$

B_n: zůstatek v měsíci n
i: měsíční úrok (např. $\frac{1}{12}$ %)
d_n: vklad v měsíci n (např. 10000 Kč)

Úročení

Zůstatek:

$$B_0 = d_0$$
$$B_n = B_{n-1} \cdot (1 + i) + d_n \text{ pro } n > 0$$

B_n: zůstatek v měsíci n
i: měsíční úrok (např. $\frac{1}{12}$ %)
d_n: vklad v měsíci n (např. 10000 Kč)

Funkce:

Úročení

Zůstatek:

$$B_0 = d_0$$
$$B_n = B_{n-1} \cdot (1 + i) + d_n \text{ pro } n > 0$$

B_n: zůstatek v měsíci n
i: měsíční úrok (např. $\frac{1}{12}$ %)
d_n: vklad v měsíci n (např. 10000 Kč)

Funkce:

```
(defun balances (deposits interest-rate)
  (lambda (month)
    (if (= month 0)
        (mem deposits 0)
      (+ (* (mem (balances deposits interest-rate) (- month 1))
            (+ interest-rate 1))
         (mem deposits month)))))
```

Hledání v posloupnosti

Posloupnost zůstatků v našem příkladě:

```
> (bind balances (balances deposits 1/1200))
#<ANONYMOUS-FUNCTION subfunction of BALANCES>

> (seq-to-list balances 20)
(10000.0 20008.334 30025.008 40050.03 50083.41 60125.15 70175.26
80233.74 90300.61 100375.87 110459.52 120551.58 130652.05 140760.94
150878.25 161003.99 171138.16 181280.78 191431.86 201591.39)
```

Za jak dlouho bude zůstatek alespoň milion?

Hledání v posloupnosti

Posloupnost zůstatků v našem příkladě:

```
> (bind balances (balances deposits 1/1200))
#<ANONYMOUS-FUNCTION subfunction of BALANCES>

> (seq-to-list balances 20)
(10000.0 20008.334 30025.008 40050.03 50083.41 60125.15 70175.26
80233.74 90300.61 100375.87 110459.52 120551.58 130652.05 140760.94
150878.25 161003.99 171138.16 181280.78 191431.86 201591.39)
```

Za jak dlouho bude zůstatek alespoň milion?

```
(first-index balances (lambda (x) (>= x 1000000)))
```